

Pre-Lecture Assignment 3 — DPG, DDPG, and Off-Policy Learning

Q1: Deterministic Policy Gradient (DPG)

$$\nabla_{\theta} V^{\pi_{\theta}}(\mu) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d_{\mu}^{\pi_{\theta}}} [\nabla_{\theta} \pi_{\theta}(s) \nabla_a Q^{\pi_{\theta}}(s, a)|_{a=\pi_{\theta}(s)}]$$

Term-by-term description:

$$\nabla_{\theta} V^{\pi_{\theta}}(\mu)$$

Gradient of the expected total discounted return w.r.t. policy parameters θ , averaged over the initial state distribution μ . This is what we want to maximize.

$$\frac{1}{1-\gamma}$$

Normalization constant arising from summing γ^t over $t=0,1,2,\dots$. It converts the unnormalized discounted state visitation measure into a proper probability distribution.

$$d_{\mu}^{\pi_{\theta}}$$

Normalized discounted state visitation distribution under the current deterministic policy π_{θ} — how often (in discounted time) each state s is visited when starting from μ .

$$\nabla_{\theta} \pi_{\theta}(s)$$

Jacobian of the deterministic policy output w.r.t. θ . Since the policy maps $s \rightarrow a$ directly (no sampling), this tells us how the action changes as we adjust each parameter.

$$\nabla_a Q^{\pi_{\theta}}(s, a)|_{a=\pi_{\theta}(s)}$$

Gradient of the action-value function Q w.r.t. the action a , evaluated at the action currently chosen by the policy. Points in the direction in action space that most increases long-run value.

Intuition: To improve the policy, at every visited state we apply the chain rule across two gradients: first, $\nabla_a Q$ identifies which direction in action space increases the Q -value; second, $\nabla_{\theta} \pi_{\theta}$ tells us how to shift the parameters θ so that the policy outputs an action closer to that better direction. Crucially, because the policy is deterministic, there is no integral over actions — the update is a pure closed-form chain rule, not a Monte Carlo estimate over actions.

Difference from Stochastic PG (P3):

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{s \sim \rho^{\pi}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi}(s, a)]$$

	Stochastic PG (P3)	DPG
Expectation over	States and actions	States only
Gradient form	Score function $\nabla \log \pi(a s)$	Chain rule $\nabla_{\theta} \pi \cdot \nabla_a Q$
Action sampling	Required (must sample $a \sim \pi$)	Not needed (closed form)
Exploration	Inherent via stochastic policy	Requires separate behaviour policy β
Scalability	Degrades in high-dim action spaces	Efficient regardless of action dim

Key insight: Stochastic PG integrates over both state and action space — variance grows with action dimensionality. DPG sidesteps the action integral entirely, computing the gradient deterministically through a single policy output.

Q2: Off-Policy Learning

Definition: Off-policy learning is a paradigm in which an agent learns about and optimizes a **target policy** π (the policy we ultimately want) while generating experience by following a different **behaviour policy** β . This decouples data collection from policy improvement, enabling the reuse of historical transitions, human demonstrations, or exploratory data that the target policy itself never produced.

Example — Robot arm with human teleoperator: A robot needs to learn a precise, deterministic grasping policy $\mu_{\theta}(s)$. Because a purely deterministic policy cannot explore on its own, a human teleoperator (the behaviour policy β) drives the arm across a wide range of configurations, generating a dataset of transitions (s, a, r, s') with broad state-space coverage. An off-policy DPG algorithm (e.g., COPDAC-Q) then updates μ_{θ} using the off-policy gradient:

$$\nabla_{\theta} J_{\beta}(\mu_{\theta}) = \mathbb{E}_{s \sim \rho^{\beta}} [\nabla_{\theta} \mu_{\theta}(s) \nabla_a Q^{\mu}(s, a)|_{a=\mu_{\theta}(s)}]$$

The transitions are drawn from ρ^{β} (the state distribution induced by the human), not from ρ^{μ} — that mismatch between the data-generating policy β and the policy being improved μ_{θ} is precisely what makes it off-policy. In the stochastic off-policy case, this mismatch requires importance-sampling corrections $\pi(a|s)/\beta(a|s)$ to correct for the action distribution shift. In DPG, the actor gradient has no integral over actions, so no importance-sampling correction is needed in the actor update — a major practical advantage.

Q3: Why DPG Suits Continuous Action Spaces

1. Eliminates the inner arg-max optimization: In Q-learning-style methods extended to continuous actions, every policy improvement step requires solving $\arg \max_a Q(s, a)$ — a separate nonlinear optimization for each state. This is computationally prohibitive when the action space is high-dimensional or continuous. DPG avoids this entirely: the deterministic policy μ_{θ} is updated by gradient ascent along $\nabla_a Q$, turning the inner optimization into a single backpropagation step through the critic.

2. Superior sample efficiency as action dimensionality grows: Stochastic PG estimates the policy gradient by sampling actions $a \sim \pi(\cdot|s)$ and computing the score-function estimator $\nabla \log \pi \cdot Q$. The variance of this estimator grows with the dimension of the action space, so exponentially more samples are needed for accurate gradient estimates in high-dimensional continuous spaces. DPG computes the gradient analytically — integrating only over the state space — so its sample complexity is independent of action dimensionality. Silver et al. (2014) demonstrated this empirically on continuous bandit tasks: DPG achieved lower regret than stochastic PG even with the same number of samples, and the gap widened as action dimensionality increased (10 $\rightarrow$ 25 $\rightarrow$ 50 dimensions).

3. Natural compatibility with function approximation (DDPG): Because μ_{θ} is differentiable and deterministic, the full gradient chain $\nabla_{\theta} \mu_{\theta}(s) \cdot \nabla_a Q^{\mu}(s, a)$ can be computed end-to-end via automatic differentiation through a neural-network actor and critic. This enables deep extensions (DDPG) that scale to complex perception-to-action tasks such as continuous robotic control and physics simulations.